

stm32f103 workspace notes

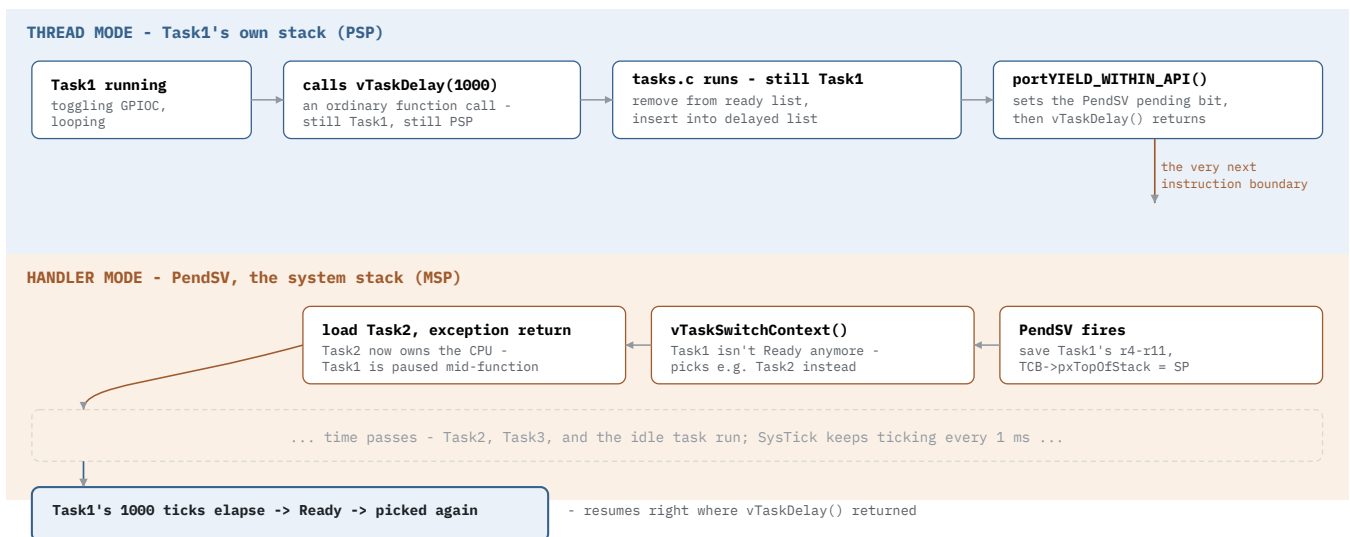
Task1: a function call vs. an interrupt

Two things that both look like "something else runs, then Task1 continues" - but only one of them actually changes the CPU's mode. Both examples below are literally what happens every time `led_task` loops in `three-thread-blink`.

1. Task1 calls into FreeRTOS, and back

`VTASKDELAY()` - A PLAIN FUNCTION CALL, UNTIL `PENDSV` TAKES OVER

Calling a FreeRTOS API isn't a mode change by itself - `vTaskDelay()` is just a C function. Task1 calls it the same way it would call any function of its own: same stack, same privilege, same everything. The mode only actually changes a few instructions later, when that call has asked the kernel to switch tasks and `PendSV` fires to carry it out.



The mode change happens at the `PendSV` boundary, not at the function-call boundary. Everything left of that line is ordinary C, running on Task1's own stack.

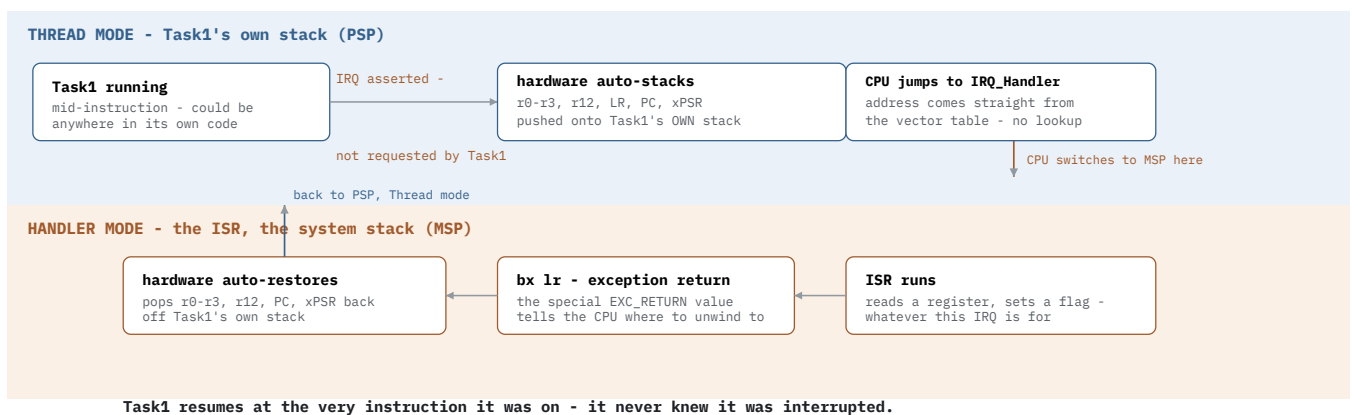
- 1 **Task1 runs normally** - the same loop body every LED task in this project has, toggling a pin and about to sleep.

- 2 **It calls `vTaskDelay(1000)`** . No trap, no mode change - this is a plain call instruction, same as calling `gpio_output_init()` .
- 3 **`tasks.c` executes, still as Task1.** It removes Task1 from `pxReadyTasksLists[]` and inserts it into `pxDelayedTaskList` , sorted by wake tick.
- 4 **The API sets PendSV pending** and returns. Task1's own code is about to resume for a moment - but only for a moment.
- 5 **PendSV fires** at the next instruction boundary, and *now* the CPU actually changes mode - Handler mode, running on the system stack, not Task1's.
- 6 **The real context switch happens here** - Task1's registers get saved to its own stack, `vTaskSwitchContext()` picks whoever's next, and that task's registers get loaded instead.
- 7 **Much later**, once 1000 ticks have passed and Task1 is Ready again, a future PendSV picks it back up - resuming precisely at the instruction after `vTaskDelay()` returned, as if no time had passed at all from Task1's point of view.

2. Task1 is interrupted, and returns

A HARDWARE IRQ - NO FUNCTION CALL, NO SCHEDULER DECISION, NO TCB TOUCHED

This is the other case: something external happens - a timer, a pin, SysTick itself - and the hardware forces the mode change, mid-instruction-stream, with no cooperation from Task1's code at all.



Nothing here touches a TCB, a ready list, or `vTaskSwitchContext()` - that machinery only runs if the ISR itself decides a switch is warranted (see the aside below).

- 1 **Task1 is running** - anywhere in its own code, at any instruction. It has no idea what's about to happen.
- 2 **The IRQ is asserted** by hardware - a peripheral, a timer, anything wired to the NVIC. Task1 did nothing to cause this.
- 3 **The CPU auto-stacks eight registers** onto Task1's *own* stack - not a separate one - then switches to Handler mode and starts using MSP for everything from here on.
- 4 **Execution jumps straight to the handler** named in the vector table at that IRQ's fixed position - no search, no dispatch table lookup.
- 5 **The ISR does its job** and ends with `bx lr`, loaded with a special `EXC_RETURN` value the hardware recognizes as "this is an exception return, not a normal branch."
- 6 **The hardware reverses step 3** automatically - pops the same eight registers back off Task1's stack, restores PSP, drops back to Thread mode.
- 7 **Task1 continues** at the exact instruction after the one it was interrupted on. No `vTaskSwitchContext()` ran; nothing about Task1's state in the scheduler's own bookkeeping ever changed.

When it isn't this simple. If the IRQ in this diagram is `SysTick`, step 5's ISR body is `xTaskIncrementTick()` - and if *that* decides a switch is due, it pends `PendSV` before returning. `PendSV` then tail-chains in immediately, *before* the hardware ever gets to step 6 - so instead of Task1 resuming, a different task's context gets loaded instead. That's the first diagram's handler-mode half, reusing this exact entry mechanism as its trigger.

The difference, side by side

CALLING FREERTOS (diagram 1)

- Starts as an ordinary function call
- Stays in Thread mode, on Task1's own PSP, through all of `tasks.c`
- Task1 *asked* for this, directly or indirectly
- Mode only changes once `PendSV` is pended and fires

A HARDWARE INTERRUPT (diagram 2)

- Starts with an asynchronous signal, not an instruction
- Switches to Handler mode and MSP immediately, by hardware
- Task1 has no say in the timing at all
- Can happen between literally any two instructions

stm32f103 workspace notes · continues from the SysTick / PendSV / SVC and task-context notes